Document Summary
*This note outlines the three stages of training neural networks: one-time setup, training dynamics, and post-training strategies. It details **model ensembles** for performance enhancement and **transfer learning** for leveraging pre-trained models, especially with CNNs. Key CNN architectures like AlexNet, VGG, GoogLeNet, and ResNet are also reviewed, highlighting their innovations and comparative complexities.*

Training Neural Networks: Overview

Training Neural Networks (NNs) involves three distinct stages, each addressing a specific aspect of model development and optimization.

1. **One-time setup**: This initial phase involves configuring fundamental components of the neural network.
   - **Activation functions**: These functions introduce non-linearity into the network, allowing it to learn complex patterns. Example: `ReLU`, `Sigmoid`, `Tanh`.
   - **Preprocessing**: Data preparation steps like normalization or standardization. Example: Subtracting the mean and dividing by the standard deviation for image pixels.
   - **Weight initialization**: Setting initial values for the network's weights. Example: Random initialization from a Gaussian distribution.
   - **Regularization**: Techniques to prevent overfitting. Example: `L2 regularization`, `Dropout`.
2. **Training dynamics**: This stage focuses on the iterative process of learning from data.
   - **Babysitting the learning process**: Monitoring metrics like loss and accuracy to ensure effective training.
   - **Parameter updates**: Adjusting network weights and biases based on gradients computed during backpropagation. Example: Using Stochastic Gradient Descent (`SGD`).
   - **Hyperparameter optimization**: Tuning parameters that control the learning process itself, such as learning rate or batch size.
3. **After training**: This final stage involves strategies to enhance the trained model's performance and applicability.
   - **Model ensembles**: Combining multiple models to improve overall prediction accuracy.
   - **Transfer learning**: Leveraging knowledge from a pre-trained model on a new, related task.

Model Ensembles

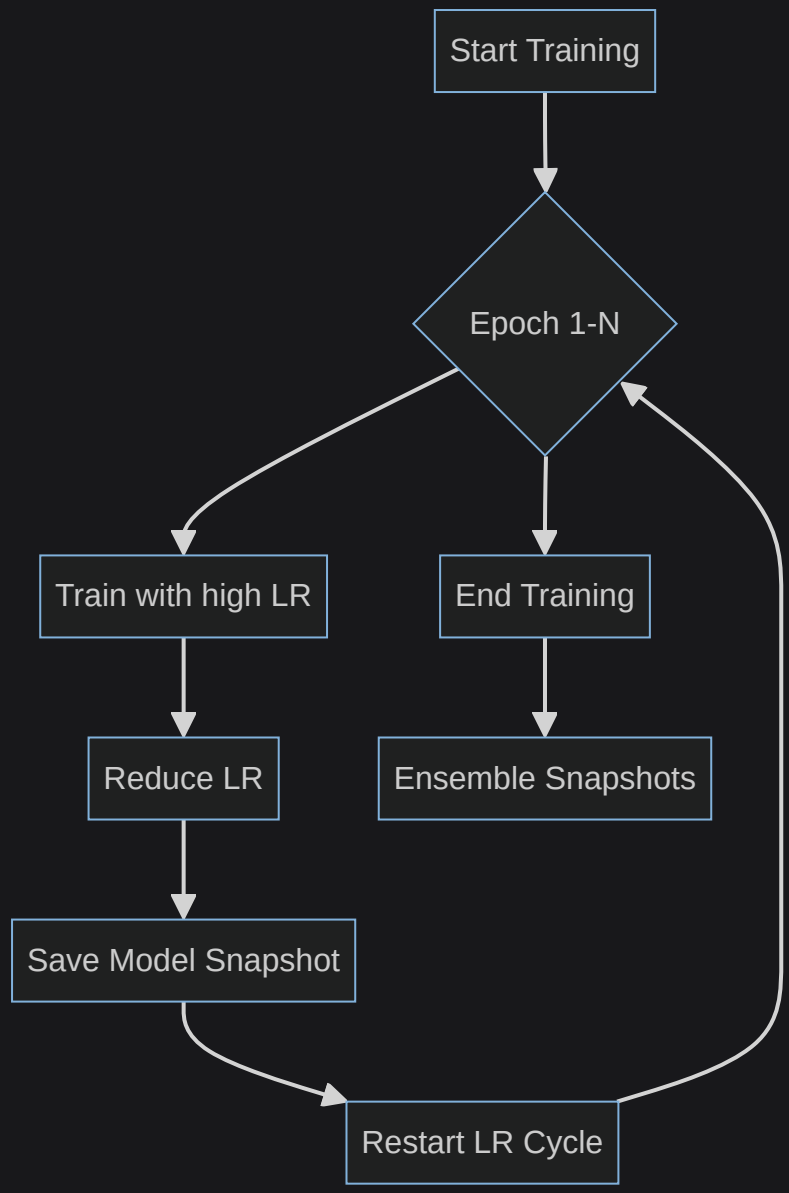
> ***Model Ensembles***
> *This technique consistently improves model performance, typically by approximately 2%. The core idea is to combine the predictions of multiple models rather than relying on a single one.*

- **Method**: The process involves training several distinct models. At test time, the predictions from these individual models are averaged to produce a final, more robust result.

Model Ensembles: Tips and Tricks

To maximize the benefits of model ensembles, consider these advanced strategies:

- **Snapshots of a single model**: Instead of training entirely independent models, a more efficient approach is to save multiple snapshots of a *single* model during its training process. These snapshots, taken at different points in training, often exhibit sufficient diversity to form an effective ensemble.
  - For deeper insights, refer to "SGDR: Stochastic gradient descent with restarts" by Loshchilov and Hutter (2016) and "Snapshot ensembles: train 1, get M for free" by Huang et al. (2017).
- **Cyclic learning rate schedules**: These schedules are particularly effective for boosting the performance of snapshot ensembles. They involve varying the learning rate in a cyclical pattern, allowing the model to explore different regions of the loss landscape and converge to diverse local minima, which are ideal for ensemble members.



- **Polyak averaging**: This technique involves using a moving average of the parameter vector (weights and biases) at test time, rather than just the final trained parameters. This averaged parameter set often leads to more stable and better-performing models.
- Reference: Polyak and Juditsky, "Acceleration of stochastic approximation by averaging" (1992).
- Example: If the parameter vector at time t is θ_t, the Polyak averaged parameter $\bar{\theta}_T$ after T steps could be $\bar{\theta}_T = \frac{1}{T} \sum_{t=1}^{T} \theta_t$.

Transfer Learning

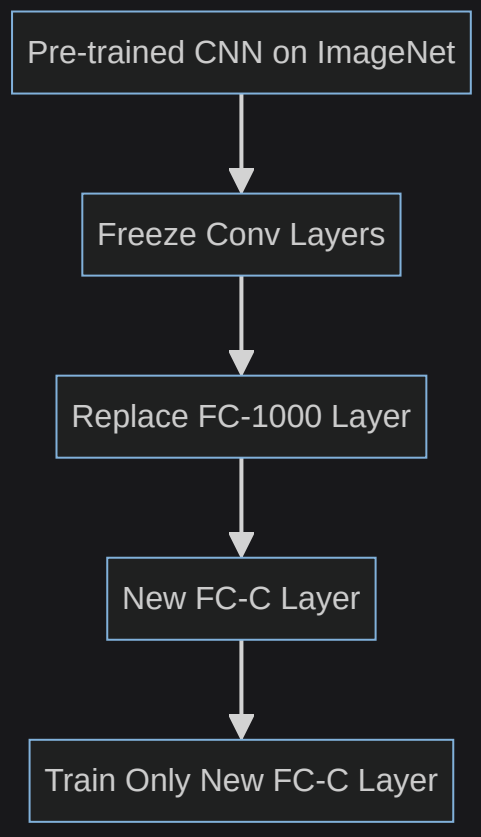
> ***Transfer Learning***
> *This powerful technique challenges the common misconception that "you need a lot of data if you want to train/use CNNs." It allows the use of convolutional neural networks (`CNNs`) even with limited datasets by leveraging pre-existing knowledge.*

Transfer Learning with CNN

Transfer learning with `CNNs` typically involves utilizing models that have been pre-trained on massive datasets like ImageNet (http://www.image-net.org/).

1. **Train on ImageNet**: First, a large `CNN` architecture, such as a `VGG`-like model, is trained from scratch on the ImageNet dataset. This dataset contains millions of images across 1000 categories, allowing the `CNN` to learn highly generalizable feature extractors.
   - **Example CNN Architecture**: `Conv-64` (convolutional layer with 64 filters), `MaxPool` (max pooling layer), `Conv-128`, `MaxPool`, `Conv-256`, `MaxPool`, `Conv-512`, `MaxPool`, `FC-4096` (fully connected layer with 4096 units), `FC-4096`, `FC-1000` (final fully connected layer for 1000 ImageNet classes).
   - References: Donahue et al., "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition" (ICML 2014); Razavian et al., "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition" (CVPR Workshops 2014).

2. **Small Dataset (C classes)**: When working with a new, small dataset (e.g., 10-100 samples per class) that is similar to ImageNet, the strategy is to:
   - **Freeze** the pre-trained convolutional layers. This means their weights are not updated during training. These layers act as fixed feature extractors.
   - **Reinitialize** and train only the final fully connected layer(s). For example, if the original ImageNet model had an `FC-1000` layer, it would be replaced with a new `FC-C` layer (where C is the number of classes in the new dataset) and only this new layer would be trained.

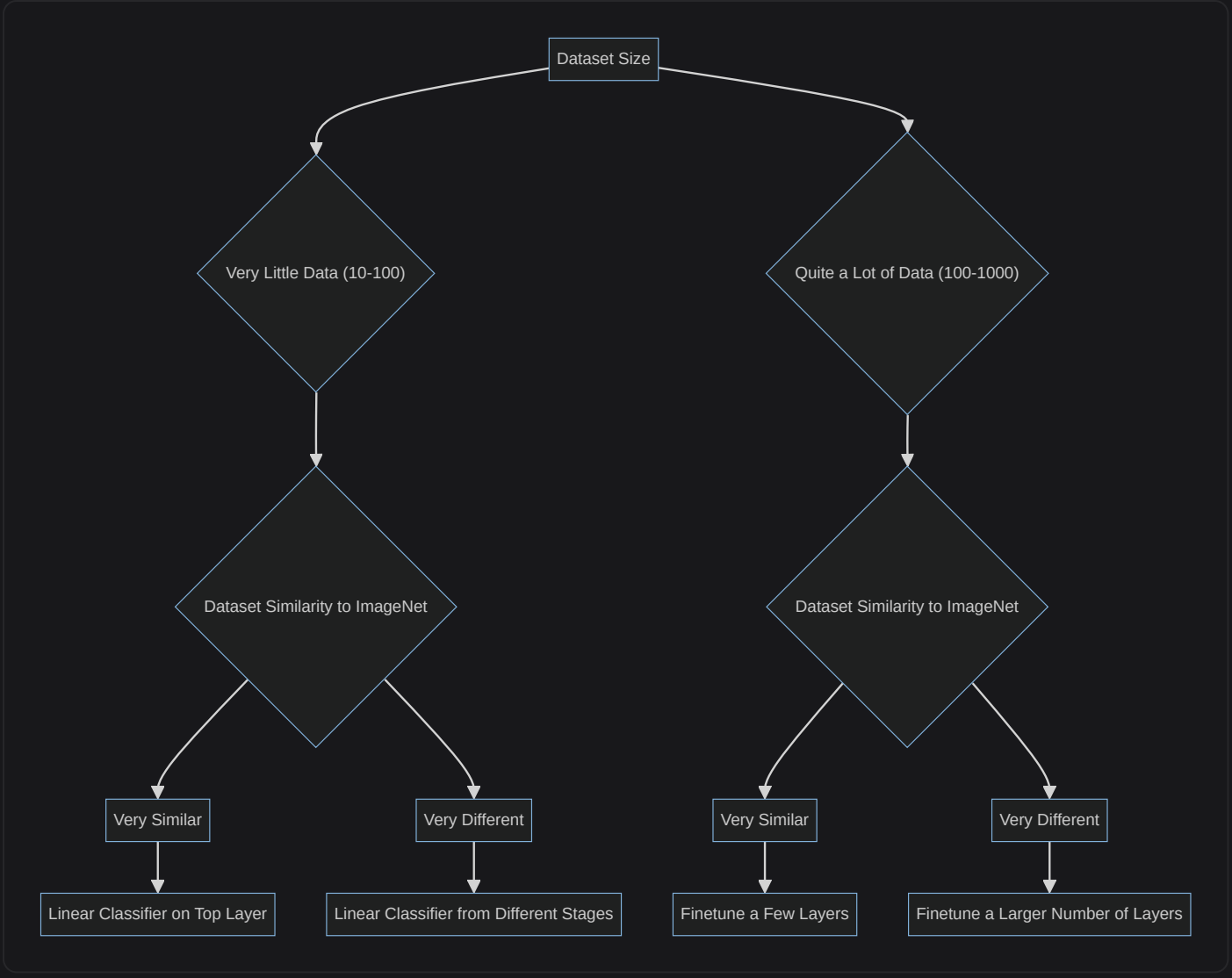


3. **Bigger Dataset: Fine-Tuning**: For larger datasets (e.g., 100-1000 samples per class), a more flexible approach called fine-tuning is used:
    - **Freeze** only the initial layers. These early layers typically learn very generic features (e.g., edges, textures) that are useful across many image tasks.
    - **Train** the later convolutional layers and the fully connected layers. These later layers learn more specific, higher-level features, and fine-tuning them allows the model to adapt these features to the new dataset.
    - **Lower learning rate**: It is crucial to use a significantly lower learning rate, often $1/10$ of the original learning rate. This prevents large weight updates that could corrupt the useful pre-trained features. Example: If the original learning rate was 0.01, use 0.001 for fine-tuning.

Transfer Learning Strategies Based on Dataset Size and Similarity

The optimal transfer learning strategy depends critically on two factors: the size of the new dataset and its similarity to the original ImageNet dataset.

- **Very similar, very little data (10-100 samples/class)**:
    - **Strategy**: Use a `Linear Classifier` on top of the pre-trained features. This means freezing all convolutional layers and training only a new, final classification layer. The pre-trained `CNN` effectively serves as a fixed feature extractor.
    - Example: Training a logistic regression model on the output of the last convolutional layer.
- **Very similar, quite a lot of data (100-1000 samples/class)**:
    - **Strategy**: `Finetune a few layers`. Unfreeze and train the later layers of the `CNN`, allowing them to adapt to the new, similar data. The early layers, which capture generic features, remain frozen.
- **Very different, very little data (10-100 samples/class)**:
    - **Strategy**: This is the most challenging scenario. It's often best to try using a `linear classifier from different stages` of the pre-trained `CNN`. Features from earlier convolutional layers might be more generic and thus more useful than highly specific features from later layers, which may not generalize well to a very different domain.
- **Very different, quite a lot of data (100-1000 samples/class)**:
    - **Strategy**: `Finetune a larger number of layers`. Unfreeze and train more convolutional layers, potentially starting from earlier in the network. With more data, the model can learn to adapt a wider range of its features to the new, distinct domain without overfitting.



Pervasiveness of Transfer Learning with CNNs

Transfer learning with `CNNs` is a cornerstone in many advanced computer vision applications:

- **Image Captioning**: This task involves generating descriptive text for an image. It often combines a `CNN` (pretrained on ImageNet for feature extraction) with a Recurrent Neural Network (`RNN`) for sequence generation (the caption).
  - **Word vectors** used in the `RNN` can also be `pretrained with word2vec` for better semantic understanding.
  - Reference: Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions" (CVPR 2015).
- **Object Detection**: Models like `Fast R-CNN` heavily rely on an ImageNet-pretrained `CNN` as their backbone for extracting features from candidate regions within an image.
  - Reference: Girshick, "Fast R-CNN" (ICCV 2015).

CNN Architectures: Case Studies

The field of Convolutional Neural Networks has seen rapid advancements, with landmark models frequently emerging from the annual ImageNet Large Scale Visual Recognition

Challenge (ILSVRC).

ILSVRC Winners: A Historical Overview

The `ILSVRC` has been a crucible for developing increasingly powerful `CNN` architectures.

- **First CNN winner**: `AlexNet` marked a significant breakthrough in deep learning for computer vision.
- **Improved AlexNet**: `ZFNet` (Zeiler and Fergus, 2013) provided architectural refinements that boosted `AlexNet`'s performance.
- **Deeper Networks**: `VGG` and `GoogLeNet` pushed the boundaries by exploring significantly deeper network structures.
- **Revolution of Depth**: `ResNet` introduced residual connections, enabling the training of extremely deep networks effectively.

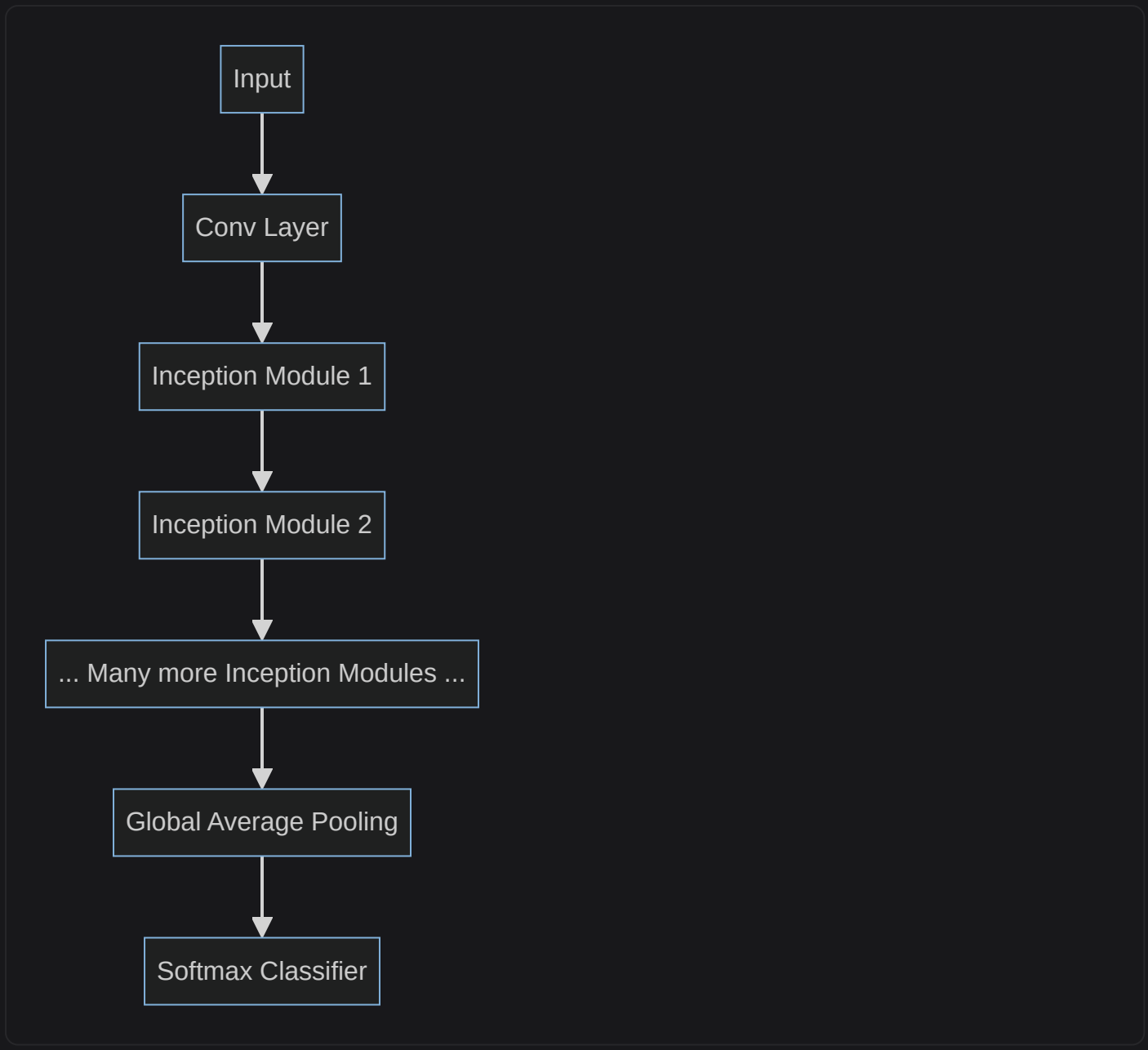
ZFNet

- **Reference**: [Zeiler and Fergus, 2013]
- **Improvements over AlexNet**: `ZFNet` made specific architectural changes to `AlexNet`, primarily in the initial convolutional layers and filter sizes of later layers.
  - The first convolutional layer (`CONV1`) was changed from a large $(11 \times 11 \text{ stride } 4)$ filter to a smaller, more detailed $(7 \times 7 \text{ stride } 2)$ filter. This allowed for finer-grained feature extraction in the early stages.
  - The number of filters in `CONV3, CONV4, CONV5` was increased from $384, 384, 256$ to $512, 1024, 512$ respectively, enhancing the network's capacity to learn complex features.
- **Performance**: These modifications resulted in a notable improvement in ImageNet top-5 error, decreasing from 16.4% (`AlexNet`) to 11.7%.

Case Study: GoogLeNet

- **Reference**: [Szegedy et al., 2014]
- **Key Features**: `GoogLeNet` introduced several innovative design principles.
  - It was significantly deeper, with `22 layers`.
  - It featured an `efficient "Inception" module`, which allowed the network to perform convolutions at multiple scales within a single block and concatenate their results. This reduced the number of parameters and computational cost.
  - Notably, it had `no FC layers` at the end, replacing them with a global average pooling layer, which further reduced parameters and helped prevent overfitting.
  - Despite its depth, `GoogLeNet` was remarkably efficient, using `only 5 million parameters` – approximately $12\times$ fewer than `AlexNet`, making it much lighter.

- **Performance**: `GoogLeNet` was the ILSVRC'14 classification winner, achieving an impressive 6.7% top-5 error.



Comparing Complexity of CNN Architectures

An analysis by Canziani, Paszke, and Culurciello (2017) provides a comparative view of the complexity and performance of various `CNN` models:

- **VGG**: This architecture is characterized by its simplicity and uniform structure (stacking `3 × 3` convolutional layers). However, it exhibits the **highest memory consumption** and requires the **most computational operations** among the compared models.
- **GoogLeNet**: Stands out for its **computational efficiency**, largely due to the Inception module design, which allows for parallel processing of filters and dimensionality reduction.
- **AlexNet**: While historically significant, it has **lower computational efficiency than GoogLeNet** and is **memory-heavy**. Its accuracy is also lower compared to more modern architectures.

- **ResNet**: Achieves **moderate efficiency** while delivering the **highest accuracy** among these models. Its key innovation, residual connections, enables the training of very deep networks by mitigating the vanishing gradient problem.
- **Inception-v4**: Represents an advanced architecture that **combines the strengths of both `ResNet` and `Inception` modules**, aiming for even higher accuracy and efficiency.

Architecture	Memory Usage	Computational Operations	Accuracy (Relative)	Key Innovation
`VGG`	Highest	Most	Moderate	Uniform `3 × 3` Convs
`GoogLeNet`	Low	Most Efficient	High	Inception Module
`AlexNet`	High	Moderate	Lower	First Deep `CNN` Winner
`ResNet`	Moderate	Moderate	Highest	Residual Connections
`Inception-v4`	Moderate	High Efficiency	Very High	`ResNet` + `Inception`